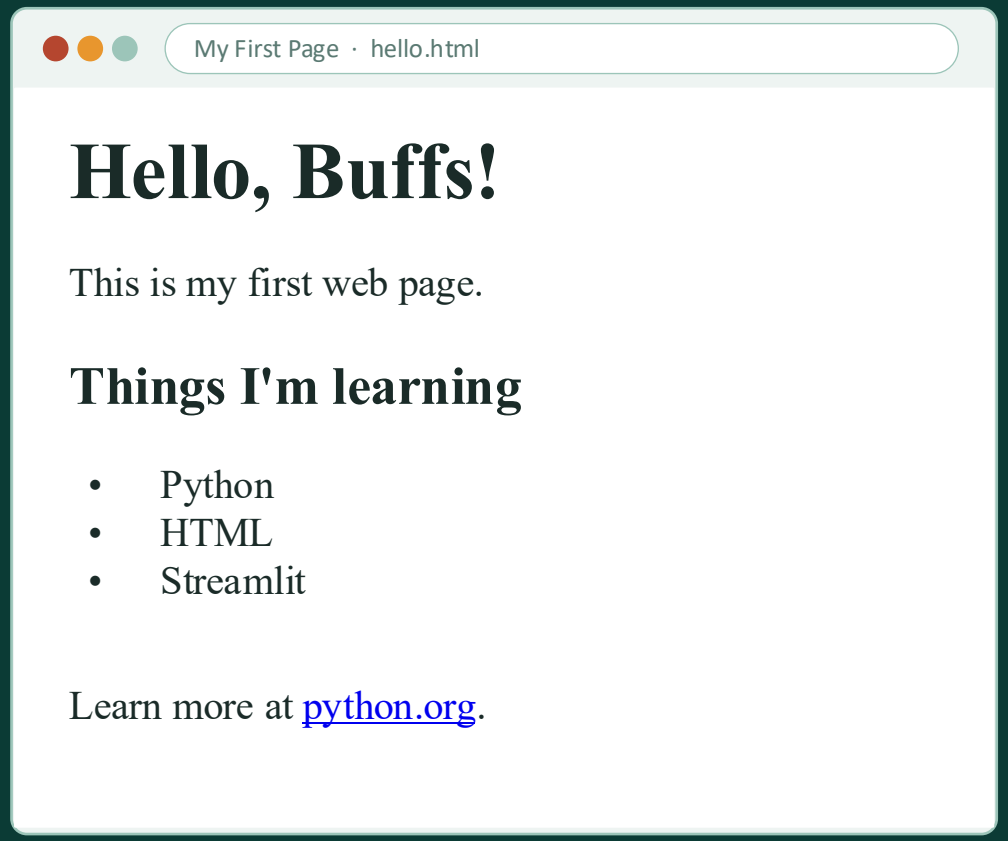
Python and HTML

From web pages to simple apps · BAIM 3220

`<h1>` `<p>` `<a>` `<ul>` `<table>`

A few of the tags you'll write today



Four parts: from a page to an app you can play



Part 1

Write a page

Tags give text structure. The browser draws it.

`hello.html`



Part 2

Let Python write it

Build HTML from data with loops and f-strings.

`render_page.py`
`sales_report.py`



Part 3

Make it interactive

Text boxes, sliders and buttons with Streamlit.

`app.py`
`click_counter.py`



Part 4

Build a game

Guess the Number, using st.session_state.

`guess_number.py`

By the end you can

- Explain what HTML tags do
- Write and open an HTML page
- Generate HTML from data with Python
- Build a simple UI with Streamlit
- Remember values with st.session_state

Set up once: one Conda environment for this module

1. **Install Miniconda**
   Graphical installer, default settings. Mac: Apple Silicon or Intel. Windows: 64-bit.

2. **Open a terminal**
   Mac: Terminal. Windows: Anaconda Prompt.

3. **Get the files**
   Download ZIP or git clone github.com/iportilla/intro-html into Downloads. Name the folder intro-html.

4. **Create the environment**
   Once: baim3220-web, with Python 3.12.

5. **Install the packages**
   Streamlit for apps, notebook for Jupyter.

```
# Check that Conda works
conda --version

# Go to the module folder (Mac)
cd ~/Downloads/intro-html

# Create once, then activate
conda create -n baim3220-web python=3.12 -y
conda activate baim3220-web

# Install the packages
python -m pip install streamlit notebook
```



Each time you come back

`conda activate baim3220-web`

Your prompt should start with (baim3220-web). In VS Code, choose it under Python: Select Interpreter.

Part 1 · HTML labels each piece of content with tags

```
<p>This is a paragraph.</p>
```

opening tag — content — closing tag (note the /)

Most tags come in pairs, with the content in between.

`<h1>` … `<h6>`
Headings, largest to smallest

`<p>`
A paragraph

`<a href="...">`
A link to another page

`<ul>` `<li>`
A bulleted list and its items

`<table>`
A table: <tr> rows, <th> header cells, <td> data cells

`<img>`
An image (src, alt). No closing tag

Part 1 · hello.html: the head describes, the body shows

```
<!DOCTYPE html>
<html>
<head>
    <title>My First Page</title>
</head>
<body>
    <h1>Hello, Buffs!</h1>
    <p>This is my first web page.</p>

    <h2>Things I'm learning</h2>
    <ul>
        <li>Python</li>
        <li>HTML</li>
        <li>Streamlit</li>
    </ul>

    <p>Learn more at
       <a href="https://www.python.org">python.org</a>.
    </p>
</body>
</html>
```



<head> holds info about the page, like the tab title. **<body>** holds what visitors see.

Try it: add your name to the heading, a 4th list item and a paragraph about your major. Save, then refresh.

Part 2 · A web page is just text, so Python can write it




```
from pathlib import Path
import webbrowser

html = """<!DOCTYPE html>
<html>
<head><title>Made with Python</title></head>
<body>
    <h1>This page was created by Python</h1>
    <p>Python wrote this HTML to a file.</p>
</body>
</html>
"""

page = Path("python_page.html")
page.write_text(html, encoding="utf-8")
webbrowser.open(page.resolve().as_uri())
```

Run it

```
python render_page.py
```

Saves python_page.html, then opens it in your browser.

Render

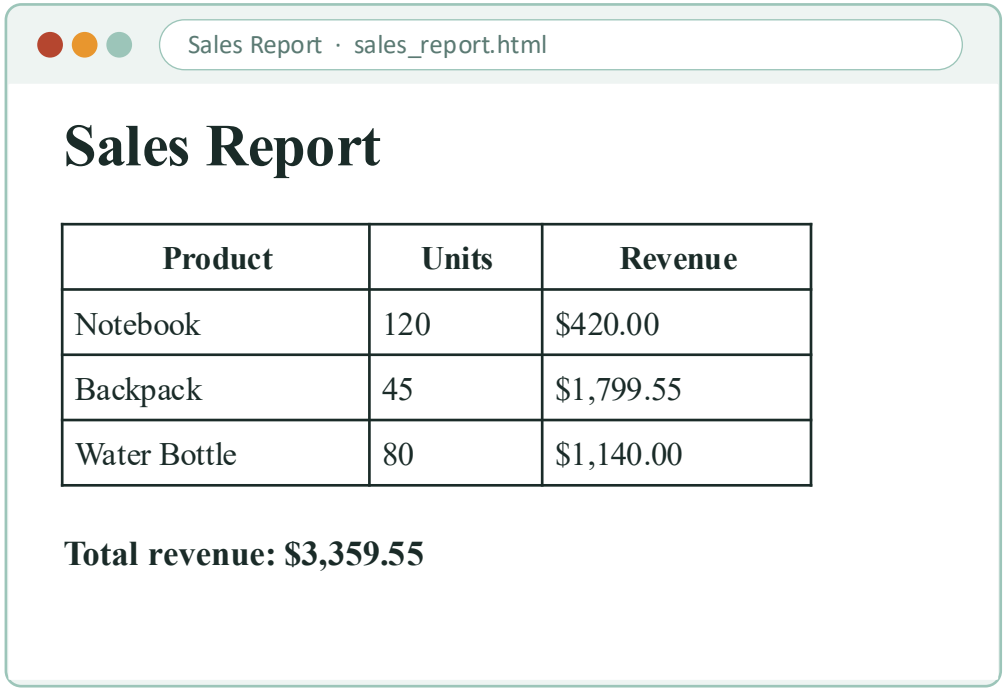
Turning tags into a formatted page. The file stays plain text: open it in VS Code to check.

Part 2 · Build a page from data: one table row per item

```
sales = [
    {"product": "Notebook", "units": 120, "price": 3.50},
    {"product": "Backpack", "units": 45, "price": 39.99},
    {"product": "Water Bottle", "units": 80, "price": 14.25},
]

rows = ""
total = 0
for item in sales:
    revenue = item["units"] * item["price"]
    total += revenue
    rows += f"""
    <tr>
        <td>{item["product"]}</td>
        <td>{item["units"]}</td>
        <td>${revenue:,.2f}</td>
    </tr>"""

# Drop {rows} and {total} into the page,
# then write sales_report.html and open it
```



- A list of dictionaries stores the data
- A for loop creates each table row
- An f-string drops the values into the HTML

Part 2 · In Jupyter, show HTML right below the cell

In [1]:

```
from IPython.display import HTML, display

display(HTML(
    "<h2>Hello from a notebook!</h2>"
    "<p>No browser tab needed.</p>"
))
```

Out:

Hello from a notebook!

No browser tab needed.



`%%writefile hello.html` at the top of a cell saves it as a file. The notebook uses it for every example.

Try it with sales_report.py

1. **Add two more products**
   Two more dictionaries in the sales list. The table grows by itself.

2. **Add a Price column**
   One more <th> in the header and one more <td> in the loop.

3. **Show the best seller**
   Add a line naming the product with the highest revenue.

Part 3 · Streamlit turns a Python script into an app

```python
import streamlit as st

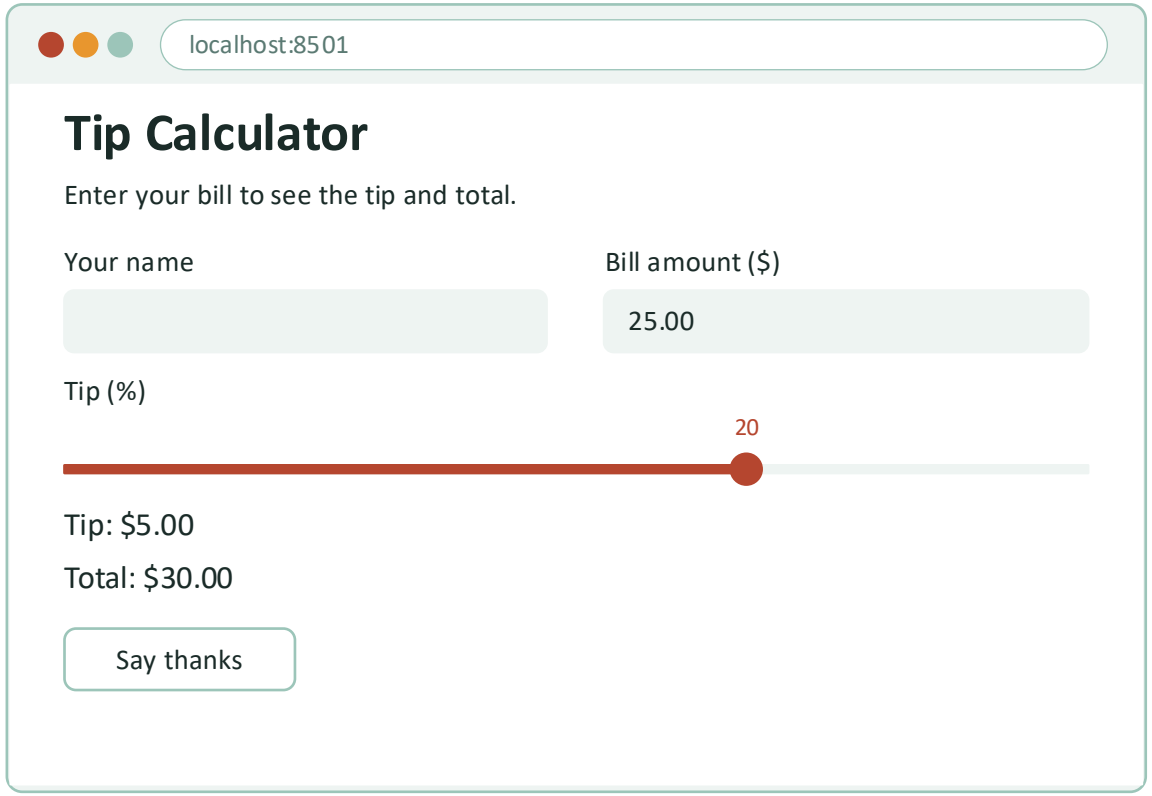
st.title("Tip Calculator")
st.write("Enter your bill to see the tip and total.")

name = st.text_input("Your name")
bill = st.number_input("Bill amount ($)",
        min_value=0.0, value=25.0, step=1.0)
tip_percent = st.slider("Tip (%)",
        min_value=0, max_value=30, value=20)

tip = bill * tip_percent / 100
total = bill + tip

st.write(f"Tip: ${tip:.2f}")
st.write(f"Total: ${total:.2f}")

if st.button("Say thanks"):
    if name:
        st.success(f"Thanks, {name}!")
    else:
        st.warning("Enter your name first.")
```




```
python -m streamlit run app.py
```

Use streamlit run, not python app.py. Ctrl+C in the terminal stops the app.

Part 3 · A handful of st. calls build most apps



`st.title("Text")`

A large page title



`st.write(value)`

Text, numbers, tables and more



`st.text_input("Label")`

A text box. Returns what was typed



`st.number_input("Label")`

A number box. Returns the number



`st.slider("Label", 0, 10)`

A slider. Returns its value



`st.button("Label")`

A button. True on the click



`st.success("Text")`

A green message



`st.warning("Text")`

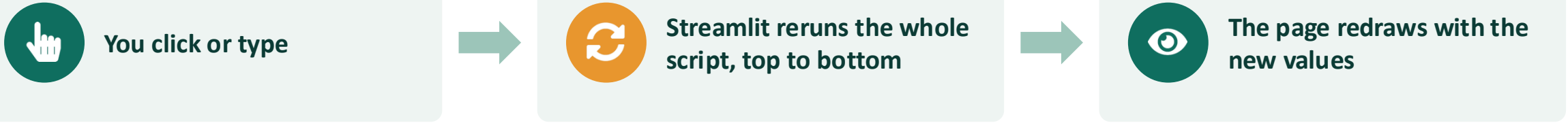
A yellow message



`st.error("Text")`

A red message

Part 3 · Every click reruns your script from the top



A regular variable starts over

Reset to 0 on every rerun

```
clicks = 0

if st.button("Click me"):
    clicks += 1

st.write(f"Clicks: {clicks}")
```

Clicks: 1 … 1 … 1

st.session_state remembers

Set once, kept between reruns

```
if "clicks" not in st.session_state:
    st.session_state.clicks = 0

if st.button("Click me"):
    st.session_state.clicks += 1

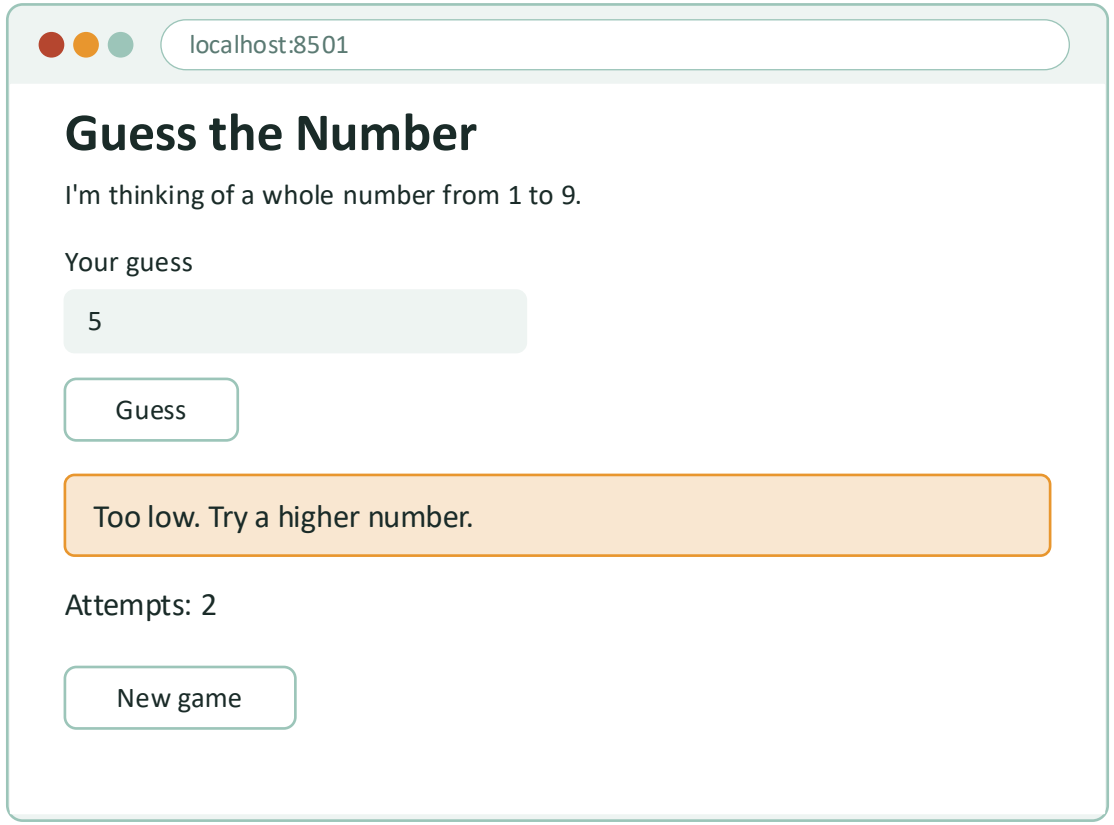
st.write(f"Clicks: {st.session_state.clicks}")
```

Clicks: 1 → 2 → 3

Part 4 · Exercise: Guess the Number

1. Pick a random whole number from 1 to 9
2. Keep the same secret until the user starts a new game
3. Let the user enter a guess from 1 to 9
4. Say whether the guess is too low, too high, or correct
5. Count the number of guesses
6. A New game button picks a new number and resets the count

- needs st.session_state to survive reruns




```
python -m streamlit run guess_number.py
```

Part 4 · Fill in five TODOs in guess_number.py

```
import random
import streamlit as st

st.title("Guess the Number")

# TODO 1: if "secret" isn't in st.session_state,
#         store a random number, set "attempts" to 0

guess = st.number_input("Your guess",
        min_value=1, max_value=9, step=1)

if st.button("Guess"):
    # TODO 2: add 1 to the attempts
    # TODO 3: "Too low", "Too high", or success
    pass

# TODO 4: show the number of attempts

if st.button("New game"):
    # TODO 5: new secret, attempts to 0, st.rerun()
    pass
```

Hints

`random.randint(1, 9)`

Includes both 1 and 9.

Copy the clicks pattern

Create the value only if it isn't in st.session_state yet.

if / elif / else

Compare guess with st.session_state.secret.

Number keeps changing?

Then it isn't stored in st.session_state.

Common mistakes and quick fixes

`conda isn't recognized`

Mac: open a new Terminal window. Windows: use Anaconda Prompt.

`No module named streamlit`

Run conda activate baim3220-web, then try again.

The app doesn't open

Start it with python -m streamlit run app.py, not python app.py.

Port 8501 is already in use

Stop the other app with Ctrl+C, or use the address the terminal prints.

The secret changes every guess

Store it in st.session_state, and only when it's missing.

The page didn't change

Save the file, then refresh the browser (or rerun the cell).

What to turn in

Submit



`guess_number.py`

your completed app



A screenshot

of your app right after a correct guess

Before you submit, check that

- ✓ The secret stays the same until New game
- ✓ Every guess says too low, too high or correct
- ✓ The attempt counter goes up, and New game resets it

Bonus challenges



Limit the player to 4 guesses



Add an easy mode and a hard mode

1 to 9, or 1 to 99



Celebrate a correct guess

`st.balloons()`



Show the player's previous guesses

a list in st.session_state

Get started

Write a page, let Python write it, then make it interactive.

Get the files

`github.com/iportilla/intro-html`

Activate

```
conda activate baim3220-web
```

Start here

`README.md` + `html_with_python.ipynb`